

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний інститут ім. Ігоря Сікорського»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №4

З дисципліни

«КРИПТОГРАФІЯ»

«Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем»

Виконали:

Студенти 3-го курсу, групи ФБ-41

Білогривий М. Р. та Мележик А. А.

Перевірив: Селюх Поліна Валентинівна

Київ 2026

Мета роботи: Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми RSA; практичне ознайомлення із системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Постановка задачі та параметри роботи:

Розглядається криптосистема RSA для двох абонентів, A і B. Необхідно реалізувати генерацію великих простих чисел (з імовірнісним тестом Міллера–Рабіна та попередніми пробними діленнями), побудувати за ними ключові пари RSA, реалізувати шифрування, розшифрування, цифровий підпис і його перевірку, а також протокол конфіденційного розсилання ключів із підтвердженням справжності відправника. Усі тести на простоту реалізовано самостійно, готові реалізації тестів не використовувались. Індивідуального варіанта в завданні немає: параметри формуються випадково.

Мова реалізації — Python 3, джерело випадкових чисел — вбудований модуль random, як того вимагає завдання. Довжина кожного з простих чисел p, q, p_1, q_1 становить 256 біт (BITS = 256), відкрита експонента $e = 2^{16} + 1 = 65537$, кількість раундів тесту Міллера–Рабіна при генерації — $k = 20$ (значення за замовчуванням), при контрольній перевірці — $k = 40$.

Програмний код складається з трьох файлів: prime.py (арифметика, тест Міллера–Рабіна, генерація простих), rsa.py (функції RSA та протокол), main.py (послідовне виконання пунктів завдання). Відповідність високорівневих процедур із завдання функціям коду наведено в таблиці.

Процедура із завдання	**Функція в коді**	**Вхідні дані**
GenerateKeyPair()	gen_keys(p, q)	p, q
Encrypt()	encrypt(m, pub)	M, (n, e) адресата
Decrypt()	decrypt(c, priv)	C, секретний ключ власника
Sign()	sign(m, priv)	M, секретний ключ відправника
Verify()	verify(m, s, pub)	M, S, (n, e) відправника
SendKey()	send_key(k, priv_a, pub_b)	k, секретний ключ A, відкритий ключ B
ReceiveKey()	receive_key(k1, s1, priv_b, pub_a)	(k_1, S_1), секретний ключ B, відкритий ключ A

Порядок виконання роботи:

1. Допоміжні арифметичні алгоритми

```python
def power(x, a, m):
    # швидке піднесення до степеня (схема Горнера)
    y = 1
    for bit in bin(a)[2:]:
        y = y * y % m
        if bit == '1':
            y = y * x % m
    return y


def gcd(a, b):
    while b:
        a, b = b, a % b
    return a


def inv(a, m):
    # обернений за модулем через розширений Евклід
    r0, r1 = a, m
    s0, s1 = 1, 0
    while r1:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        s0, s1 = s1, s0 - q * s1
    if r0 != 1:
        raise ValueError("оберненого немає")
    return s0 % m
```

Функція power реалізує швидке піднесення до степеня за модулем за схемою Горнера: показник a записується у двійковій системі, і для кожного його біта виконується піднесення до квадрата за модулем, а для одиничного біта — ще й множення на x за модулем. Для показника довжиною k біт потрібно не більше k піднесень до квадрата та k множень за модулем. Усі проміжні значення зводяться за модулем, тому їхній розмір не перевищує розміру m.

Функція gcd обчислює найбільший спільний дільник алгоритмом Евкліда. Функція inv знаходить обернений елемент за модулем розширеним алгоритмом Евкліда: на кожному кроці виконується рівність $r_i = s_i a + t_i m$, і якщо останній ненульовий залишок дорівнює 1, то $a^{-1} \equiv s \pmod{m}$; інакше обернений елемент не існує, і викидається виняток.

2. Тест Міллера–Рабіна, пробні ділення та генерація простого числа

```python
def trial(n):
    for p in small:
        if n % p == 0:
            return n == p
    return True
```

Перед основним тестом число перевіряється пробними діленнями на 25 найменших простих (від 2 до 97) — функція trial. Це швидко відкидає значну частину складених кандидатів (див. експеримент у п. 3).

```python
def miller_rabin(n, k=20, log=False):
    if n < 2:
        return False
    if n in small:
        return True
    if not trial(n):
        return False

    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    if log:
        print(f"n = {n}\nn-1 = d * 2^s, s = {s}, d = {d}")
```

```python
    for i in range(k):
        x = random.randrange(2, n - 1)
        if gcd(x, n) != 1:
            if log:
                print(f"основа {x}: gcd != 1, складене")
            return False

        y = power(x, d, n)
        if log:
            print(f"\nоснова x = {x}\nx^d mod n = {y}")
        if y == 1 or y == n - 1:
            if log:
                print("-> сильно псевдопросте за цією основою")
            continue

        ok = False
        for r in range(1, s):
            y = y * y % n
            if log:
                print(f"r = {r}: {y}")
            if y == n - 1:
                ok = True
                break
            if y == 1:
                break
        if not ok:
            if log:
                print("-> не сильно псевдопросте, складене")
            return False
        if log:
            print("-> сильно псевдопросте за цією основою")
    return True
```

Функція miller_rabin подає $n - 1 = d \cdot 2^s$ з непарним d та виконує k раундів. У кожному раунді випадково обирається основа x з інтервалу $[2, n - 2]$; якщо $\gcd(x, n) \neq 1$, то n складене. Інакше обчислюється $y = x^d \bmod n$; якщо $y = \pm 1$, n є сильно псевдопростим за основою x. В іншому разі y послідовно підноситься до квадрата ($r = 1, \ldots, s - 1$): якщо отримано -1, число сильно псевдопросте за основою x, якщо отримано 1 — число складене. Якщо жодна умова не виконалась, n складене. Тест односторонній: для простого n він завжди відповідає «просте», а для складеного помиляється з імовірністю не більше 1/4 на раунд, тому після k раундів імовірність помилки не перевищує 4^{-k}: при $k = 20$ це приблизно $9{,}1 \cdot 10^{-13}$, при $k = 40$ — приблизно $8{,}3 \cdot 10^{-25}$. Параметр log вмикає виведення проміжних результатів.

```python
def gen_prime(bits, bad=None, log=False):
    # bad - список, куди складаємо кандидатів, що не пройшли тест
    while True:
        n = random.getrandbits(bits) | (1 << (bits - 1)) | 1
        if miller_rabin(n, log=log):
            return n
        if bad is not None:
            bad.append(n)
```

Функція gen_prime генерує випадкове число заданої довжини, у якого старший біт (забезпечує довжину рівно 256 біт) і молодший біт (забезпечує непарність) встановлені в 1, і повторює спробу, доки кандидат не пройде тест. Кандидати, що не пройшли тест, накопичуються у списку bad.

Проміжні результати тесту Міллера–Рабіна (одне з простих чисел, одна основа, $k = 1$; значення в десятковій системі):

Параметр	Значення (десяткова система)
n	725304273423279200859792558273706712201214316968793783184169216095453099869 61
$n - 1 = d \cdot 2^s$	s = 4; d = 453315170889549500537370348921066695125758948105496114490105760059658187418 5
основа x	126690236956036422287543110464151887064651758547710444723905267585200438086052
$x^d \bmod n$	255705286236182637412839899595888435245428826002755925201769589052678689298595
$r = 1$: $y^2 \bmod n$	725304273423279200859792558273706712201214316968793783184169216095453099869 60

Тут $s = 4$, а $x^d \bmod n$ не дорівнює ± 1, тому виконується цикл за r. Уже при $r = 1$ отримано значення, що дорівнює $n - 1$ (відрізняється від n останньою цифрою: …961 і …960), тобто -1 за модулем n. Отже, n є сильно псевдопростим за основою x, цикл завершується достроково.

Для наочності додатково розглянуто складене число, яке пройшло пробні ділення, але було відкинуто тестом (окремий скрипт, значення в десятковій системі):

Параметр	Значення (hex)
n	9509706575404871249301946223082776852361632420885369174343575412 8633565197369
d, s	s = 3; d = 11887133219256089061627432778853471065452040526106711467929469266079195649671
основа x	5231860855471731500352663007378773303642802683963509888446275581 4501044397423
$x^d \bmod n$	7730291324195180423697199058087124993942451547290113218978655556 0063676126639
$r = 1$	6887628415047804607121594354863161503226093422131781674896134442 9661823320791
$r = 2$	8689638284832162654107495238955416076159836280388059878773875262 8002497913186

Тут $s = 3$, тому перевіряються $r = 1$ та $r = 2$. Жодне зі значень x^d, x^{2d}, x^{4d} не дорівнює ± 1 за модулем n, отже n не є сильно псевдопростим за основою x і є складеним.

3. Генерація пар простих чисел

```
def pair(bad):
    p = gen_prime(BITS, bad)
    q = gen_prime(BITS, bad)
    while q == p:
        q = gen_prime(BITS, bad)
    return p, q
```

```
bad = []
p, q = pair(bad)
p1, q1 = pair(bad)
if p * q > p1 * q1:
    p, q, p1, q1 = p1, q1, p, q
```

Функція pair генерує два різні прості числа (випадок $q = p$ відкидається). Для абонентів A і B генеруються дві пари, після чого пари впорядковуються так, щоб $pq \le p_1 q_1$. Ця умова гарантує $n \le n_1$, що необхідно для протоколу розсилання ключів: підпис $S < n$ має бути коректно зашифрований за модулем n_1.

Отримані прості числа (hex), довжина кожного — 256 біт:

Параметр	Значення (hex)
p	903c431fe0f9d5cdba0d7edad65146f22f2ca2e3dfe11f05d2e3ef31cd1735bd
q	e85ea96b54bfeef19903700088ab65111ba39f32c794fc771cf62a4f3310838b
p_1	e4eafe95c418a3cdcd8b2adf754d7302045759d58a5c345807d5aebff4f9621b
q_1	f63ac30119f8e65db7a71afd55d6b2343dc9cf81519d39ce35bffea562668889

Під час генерації чотирьох чисел було відкинуто 385 кандидатів, що не пройшли тест на простоту. Перші п’ять із них (hex):

Параметр	Значення (hex)
1	81c5706b0df82c369a1e4aa904062d1652d262e99e7dc10abaeb40b5f3a6af87
2	8d854b197500aee8ddbe1d3ee854b8e7e05b0dd123f8f2c3aac666635d5ad097
3	d5e8e98c516c775c0af391b59e251acfa0ab5eb6fa0adce750e768883f4c3711
4	90813ee4d36e57065b75c02422451720173404 6b923b4e3f514c9fa8f92b2a5d
5	87b61f742a6ebd12b501f0c0861b91fd6accff82e3707a97630d9fa9e57c4921

Додатково простоту всіх чотирьох чисел перевірено зовнішньою утилітою openssl prime, яка її підтвердила. Це лише контроль; у самій програмі використано власну реалізацію тесту.

Експеримент зі статистики відбору. Для оцінки ефективності відсіву було згенеровано 300 простих чисел довжиною 256 біт (окремим скриптом stats.py з фіксованим початковим значенням генератора для відтворюваності). Результати наведено в таблиці.

Показник	Значення
Кількість згенерованих простих	300
Усього кандидатів перевірено	27013
Кандидатів на одне просте (середнє)	90.0
Очікувана кількість $\approx \ln(2^{256})/2$	$\approx 88{,}7$

Показник	Значення
Відкинуто пробними діленнями	20519 (76,0 %)
Відкинуто тестом Міллера–Рабіна	6194
Середній час на одне просте число	7,9 мс

Густина простих чисел поблизу 2^{256} дорівнює $1/\ln(2^{256}) \approx 1/177{,}4$, а серед непарних чисел — удвічі більша, тому для знаходження одного простого потрібно в середньому ≈ 88,7 непарних кандидатів; експериментальне значення (90,0) збігається з очікуваним. Частка кандидатів, що відсіюються пробними діленнями на непарні прості до 97, теоретично дорівнює $1 - \prod(1 - 1/p) \approx 75{,}9$ %, що відповідає виміряним 76,0 %. Отже, приблизно три чверті кандидатів відкидаються без жодного піднесення до степеня, і на дорогий тест Міллера–Рабіна потрапляє лише чверть кандидатів. У основному запуску на чотири числа припало 385 відкинутих кандидатів (≈ 96 на число), що узгоджується з цими оцінками.

4. Генерація ключових пар RSA

```
def gen_keys(p, q):
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 2 ** 16 + 1
    while gcd(e, phi) != 1:
        e = random.randrange(2, phi - 1)
    d = inv(e, phi)
    return (n, e), (d, p, q)
```

Функція gen_keys обчислює $n = pq$ та $\varphi(n) = (p - 1)(q - 1)$. Відкрита експонента $e = 2^{16} + 1$ береться за замовчуванням; якщо $\gcd(e, \varphi(n)) \neq 1$, обирається випадкове e з потрібною властивістю. Секретна експонента знаходиться як $d = e^{-1} \bmod \varphi(n)$ розширеним алгоритмом Евкліда. Функція повертає відкритий ключ (n, e) та секретний ключ (d, p, q).

Ключі абонента A, побудовані за парою (p, q), та абонента B, побудовані за парою (p_1, q_1) (hex):

Параметр	Значення (hex)
n	82ebf269cad708f398ab556129ac196adcbc946b29af74d9d5bbc236c5bebd6d9583853bf4533b2594b949367ed2c6c48688b545cc4f0b076cba64783ee9e49f
e	10001

Параметр	Значення (hex)
d	2dae4c1813b9721b24f2eb488ab4cced97758947d6f4ac8df160a104617d0b34 f4e54799abf779f376075af79ea52312b756c581f6966706684a0e7e347ca589
n_1	dc2e5c41946eb273989187f8f2d7db246f05356f5836cefd9929fdbb0605f2ab 6a08e9b9856f6951e52d3b12d93bc1858d77e14621592630efab97310255d873
e_1	10001
d_1	7561f5e73475eb873b1063183287279a7de19e02355a4af1e24503b3405bcd69 ddf676803b100d968cd6e301a573aaee8f30b530c7a06722b49c801f078ec2f1

Для ключів абонента A програма перевіряє співвідношення, необхідні для коректної роботи RSA:

Перевірка	Результат
p, q — прості (Міллер–Рабін, $k = 40$)	True, True
$n = p \cdot q$	True
$\gcd(e, \varphi(n)) = 1$	True
$e \cdot d \equiv 1 \pmod{\varphi(n)}$	True

5. Шифрування, розшифрування, цифровий підпис та його перевірка

```
def encrypt(m, pub):
    n, e = pub
    return power(m, e, n)


def decrypt(c, priv):
    d, p, q = priv
    return power(c, d, p * q)


def sign(m, priv):
    d, p, q = priv
    return power(m, d, p * q)


def verify(m, s, pub):
    n, e = pub
    return power(s, e, n) == m
```

Кожну операцію реалізовано окремою функцією, на вхід якої подаються лише потрібні для неї дані: Encrypt ($C = M^e \bmod n$) використовує тільки відкритий ключ адресата; Decrypt ($M = C^d \bmod n$) і Sign ($S = M^d \bmod n$) — лише секретний ключ власника; Verify перевіряє рівність $M = S^e \bmod n$ за відкритим ключем відправника.

Відкрите повідомлення M обрано датчиком випадкових чисел з інтервалу [1, $n - 1$] (hex):

Параметр	**Значення (hex)**
M	612c55237347d58a51f022c19b97f86e544e5325f87c0c5919390b538ea5a39c f504b6a9456ea9c1f6041832229a80102f994cde8104589238f133e438c72b3f

Результати для абонентів A та B (hex):

Параметр	**Значення (hex)**
C (A)	3ceda471165601d7975e5930b1f2b7e64dbb11b49ea76d365fdf20e90b2c8a6f 2d0ca07b6678a71ee1a660494c9097220f346a866b97b7e823a8802f041ec8d7
S (A)	76434ddcbc154c6719b3acffd166f27fbe31a15695c8413484613cd8b117915e 0a5c4fa093ec40e901bf9996955b9b2fbb714152d567807721898e37b0bfacc9
C (B)	a16ea505058b69a9877a99b3756232e1f3177cd06e981de7fa86f066a2a244d7 8119a54040add85c3ab88f03988fe2c887dc13054d9b6a06908c0e37df740aab
S (B)	9b4af89c8d4d4acfd3ebb677cd393f5e19abf4bd46bff022d17e384c76c8ad42 8194c0c86962bb9f8f46656bff187bea23314aaf1845f2156ade69b94eae4898

Для обох абонентів розшифрування повернуло вихідне M (розшифровано вірно: True), а перевірка цифрового підпису підтвердила його справжність (підпис вірний: True). Для абонента A шифрування виконується ключем (n, e), для B — ключем (n_1, e_1); повідомлення з підписом (M, S) складається та перевіряється тим самим способом.

6. Протокол конфіденційного розсилання ключів із підтвердженням справжності відправника

```python
def send_key(k, priv_a, pub_b):
    k1 = encrypt(k, pub_b)
    s = sign(k, priv_a)
    s1 = encrypt(s, pub_b)
    return k1, s1


def receive_key(k1, s1, priv_b, pub_a):
    k = decrypt(k1, priv_b)
    s = decrypt(s1, priv_b)
    if not verify(k, s, pub_a):
        return None
    return k
```

Абонент A передає абоненту B секретне значення k. Перед початком роботи необхідно, щоб $n_1 \geq n$; це гарантується вибором пар $pq \leq p_1 q_1$. Послідовність дій наведено в таблиці (всі значення в hex).

Крок	Хто	Дія	Ключ
1	A	Обирає випадкове $0 < k < n$	—
2	A	$k_1 = k^{e_1} \bmod n_1$	відкритий ключ B
3	A	$S = k^d \bmod n$	секретний ключ A
4	A	$S_1 = S^{e_1} \bmod n_1$	відкритий ключ B
5	A → B	Надсилає (k_1, S_1) по відкритому каналу	—
6	B	$k = k_1^{d_1} \bmod n_1$; $S = S_1^{d_1} \bmod n_1$	секретний ключ B
7	B	Перевіряє $k = S^e \bmod n$	відкритий ключ A

Числові значення одного виконання протоколу (hex):

Параметр	Значення (hex)
k	3c9b2f85a2ed1f0495b434631d147e801b61094280d762039337ae6b77bb56d7 db9cf78c27ff4fdfe98ceac33f60a9197e68ea3239df6de36a812dbec3d93fe5
k_1	bd6724c41357c02870d88a05386442fd89d532c738d256e453edbc1155037dbb 2b82866e84e0dea5ba573ea8fe05fcedec6032b09080a8349bb1aa47f89d38d3
S_1	45a22b173664baeb565ec54ecd97d688d3923d7054068c5cd0fb07f26d4e77bb 52d1d73a0d1fe8b4922d7bf9f65a2c4d94976ca103fed2cdb8490c10e0f5bc10

Абонент B отримав значення k, що збігається з надісланим (збігається: True), підпис пройшов перевірку. Конфіденційність забезпечується шифруванням на відкритому ключі B (e_1, n_1): розшифрувати k_1 та S_1 може лише власник d_1. Підтвердження справжності відправника забезпечується підписом $S = k^d \bmod n$, який можна створити лише за секретним ключем A і перевірити за відкритим ключем A. Підпис також передається в зашифрованому вигляді (S_1), щоб не розкривати його стороннім спостерігачам.

Додатково (скриптом verify.py) перевірено поведінку протоколу при спотворенні даних: якщо змінити S_1 на одиницю, або перевіряти підпис не за ключем A, функція receive_key повертає None, тобто ключ відхиляється.

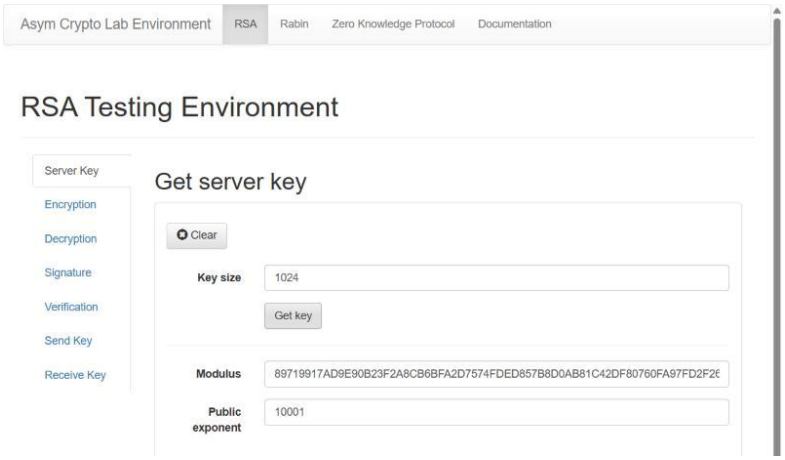
7. Перевірка взаємодії з тестовим середовищем

Взаємодію програми з тестовим середовищем (http://asymcryptwebservice.appspot.com/?section=rsa) перевірено в браузері. Сервер генерує власну ключову пару заданої довжини та виконує операції RSA; повідомлення, шифртексти, підписи та ключі передаються в шістнадцятковій системі (режим Bytes). Власні значення для перевірок обчислювались функціями з rsa.py на ключах абонентів A і B із пунктів 4–6. Повні значення наведено в таблицях, а на скріншотах довгі поля частково обрізані шириною елементів вікна.

Протокол розсилання ключів вимагає, щоб модуль відправника не перевищував модуль одержувача. Тому для перевірок, де сервер є одержувачем (ReceiveKey), використано серверний ключ 1024 біт (> 512 біт модуля A), а для SendKey, де сервер є відправником, згенеровано новий серверний ключ 256 біт (< 512 біт модуля B).

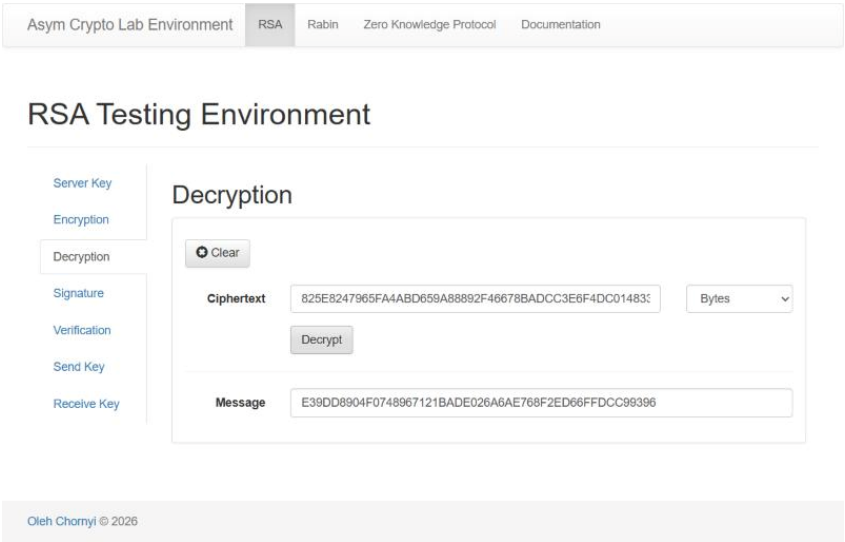
№	Перевірка	Результат
1	Encrypt власною реалізацією (відкритий ключ сервера) → Decrypt на сервері	повідомлення збіглося
2	Encrypt на сервері (відкритий ключ A) → Decrypt власною реалізацією	повідомлення збіглося
3	Sign на сервері → Verify власною реалізацією	True (для зміненого M — False)
4	Sign власною реалізацією → Verify на сервері	true
5	SendKey власною реалізацією (A) → ReceiveKey на сервері	ключ отримано, перевірка true
6	SendKey на сервері → ReceiveKey власною реалізацією (B)	ключ отримано; з хибним ключем — відхилено

7.1. Ключ сервера (1024 біти). Запит «Get key» повертає модуль та відкриту експоненту e = 10001 (hex).



Параметр	Значення (hex)
n_s	89719917AD9E90B23F2A8CB6BFA2D7574FDED857B8D0AB81C42DF80760FA97FD 2F26C08701553A6CC2F7105B5194B9CA205E776E494B1F2ED96E94734492D817 1C929E270D56DBB8386189A12A56A1C749D45518CE1F3FC19F3BC48094E7C306 7D5561068BD068DDAEC1DB5D04B3F14AF01198BC92BF4BEEE55EB8740351393B
e_s	10001

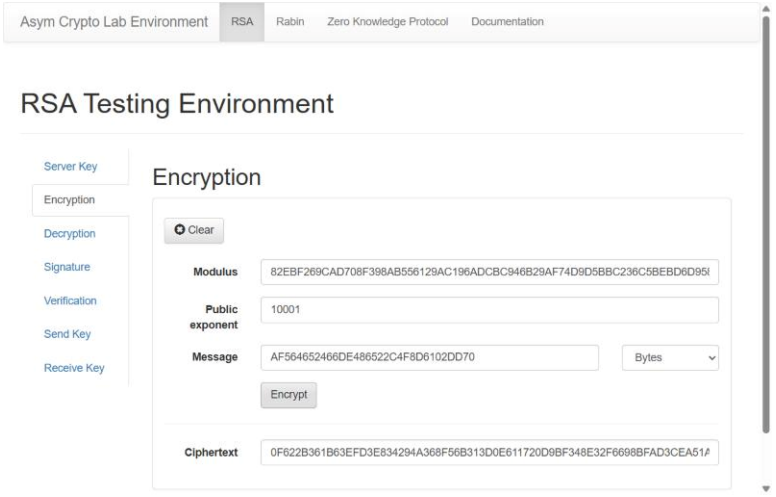
7.2. Encrypt власною реалізацією → Decrypt на сервері. Повідомлення M зашифровано функцією encrypt за відкритим ключем сервера (n_s, e_s); шифртекст C подано на вхід вкладки Decryption.



Параметр	Значення (hex)
M	E39DD8904F0748967121BADE026A6AE768F2ED66FFDCC99396
C	825E8247965FA4ABD659A88892F46678BADCC3E6F4DC014833 42979C18E7D49E 3F9C02A8B914330A28268DC0692AE65942B01F0698F294AD4D AF5A74C64C667A AA94E3939850AFF82108FEC4A1E8CA17CC00A0F1D9F5192956 C8B23DA1896B6D 9962D289D1BAE52809E836643CE64BF166587488FB843623A5E 0ACAFECC6A035
Відповідь сервера	E39DD8904F0748967121BADE026A6AE768F2ED66FFDCC99396

Сервер повернув значення, яке збігається з вихідним повідомленням *M*.

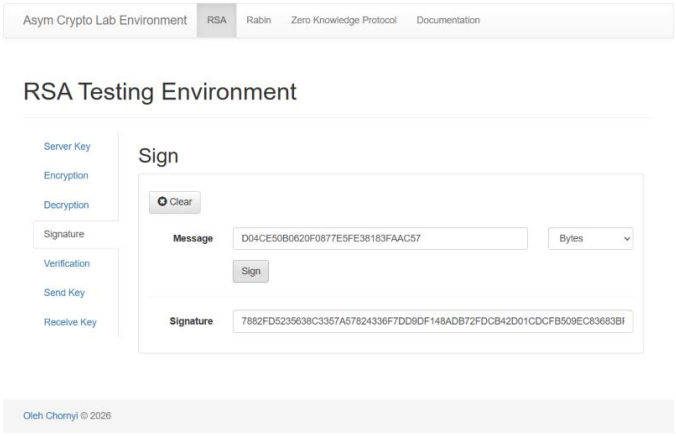
7.3. Encrypt на сервері → Decrypt власною реалізацією. На вкладці Encryption задано відкритий ключ абонента A (*n*, *e*) із п. 4 та повідомлення *M*; отриманий шифртекст розшифровано функцією decrypt за секретним ключем *d* абонента A.



Параметр	Значення (hex)
M	AF564652466DE486522C4F8D6102DD70
C (сервер)	0F622B361B63EFD3E834294A368F56B313D0E611720D9BF348E 32F6698BFAD3C EA51A397439F5E0DF26EB384CF7BF9DD8A4AEE703ECC8D845 789DE6A4F6A9CD8
decrypt(*C*)	AF564652466DE486522C4F8D6102DD70

Локальне розшифрування повернуло вихідне повідомлення (порівняння з *M* дало True).

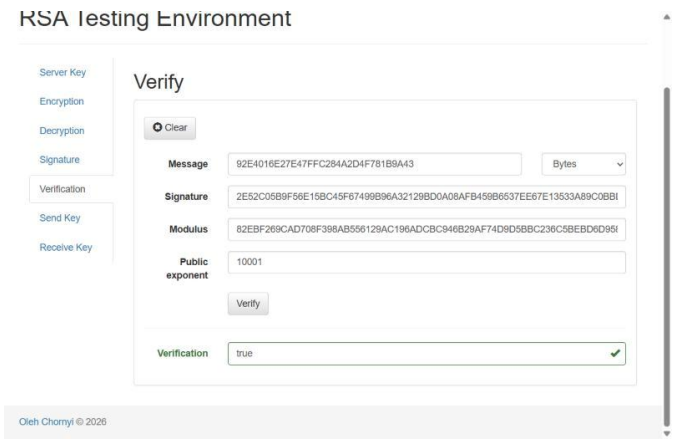
7.4. Sign на сервері → Verify власною реалізацією. Сервер підписав повідомлення *M* своїм секретним ключем (ключ 1024 біти); підпис *S* перевірено функцією verify за відкритим ключем сервера.



Параметр	Значення (hex)
M	D04CE50B0620F0877E5FE38183FAAC57
S (сервер)	7882FD5235638C3357A57824336F7DD9DF148ADB72FDCB42D01CDCFB509EC836 83BFD82D5334744E46AF15EA5DB066DDE2B246585F45E989F860303F898FA5B7 DF7485B4B9EA874D45AD1294DBFDBF322DD9124717CDE145E2DC595390AF9C42 DDAD4B3EB54C1BED4637A753D1E08D41C46318399C32A6865BEC2363F15584CA

Функція verify повернула True. Для контролю повідомлення було змінено на *M* + 1: перевірка повернула False.

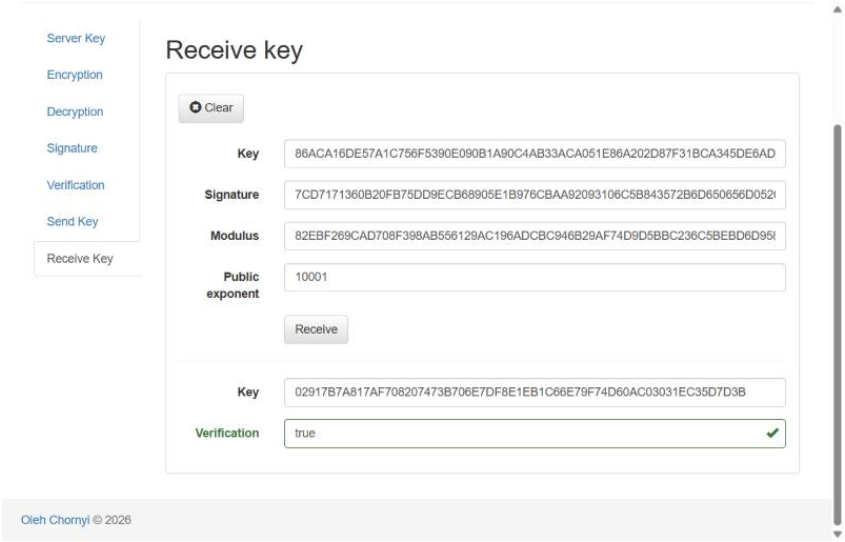
7.5. Sign власною реалізацією → Verify на сервері. Повідомлення *M* підписано функцією sign секретним ключем абонента A; на вкладці Verification введено *M*, *S* та відкритий ключ A (*n*, *e*).



Параметр	Значення (hex)
M	92E4016E27E47FFC284A2D4F781B9A43
S (A)	2E52C05B9F56E15BC45F67499B96A32129BD0A08AFB459B6537EE67E13533A89 C0BBD360FF8C01AE554E88240B52F28AC81440DC7DEB4608B6CB83C19DDA6D75

Сервер повернув результат перевірки true.

7.6. Протокол: SendKey власною реалізацією (A) → ReceiveKey на сервері. Абонент A обрав ключ $0 < k < n$ та сформував пару (k_1, S_1) функцією send_key за відкритим ключем сервера (1024 біти). Пару та відкритий ключ A введено на вкладці Receive Key.

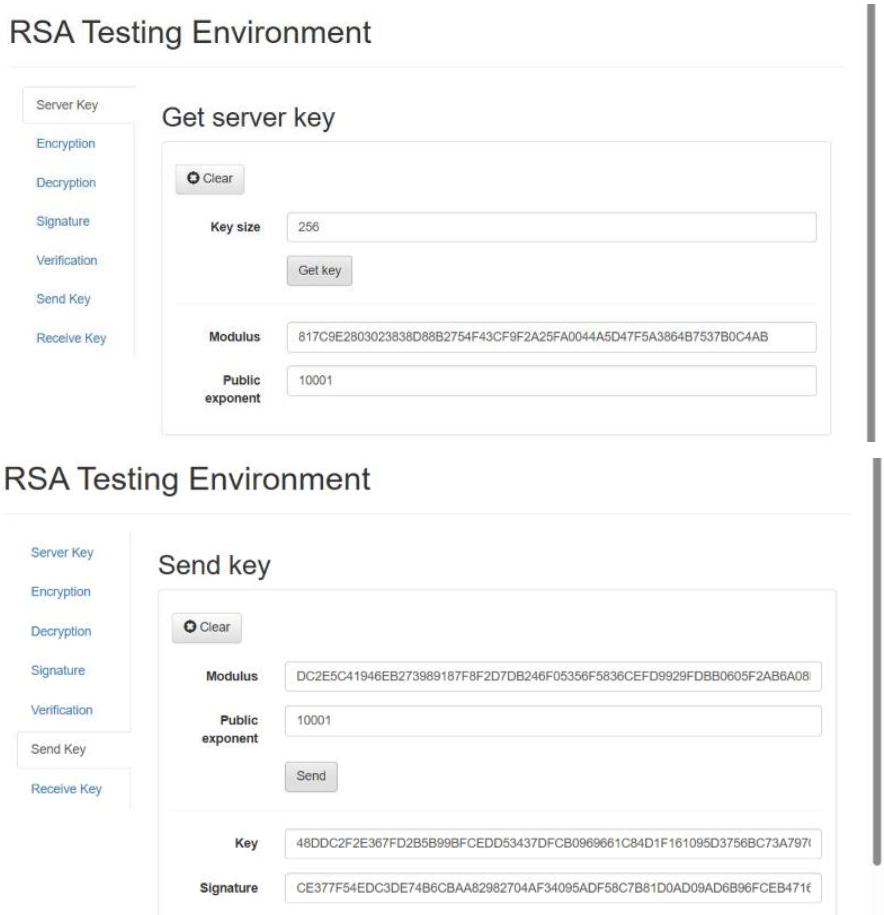


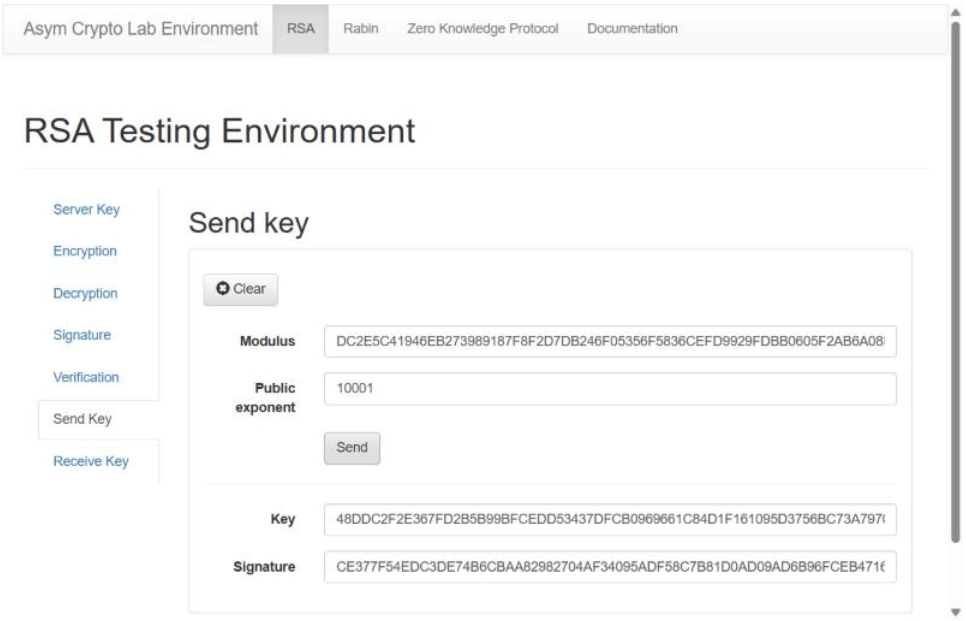
Параметр	Значення (hex)
k	2917B7A817AF708207473B706E7DF8E1EB1C66E79F74D60AC03031EC35D7D3B
k_1	86ACA16DE57A1C756F5390E090B1A90C4AB33ACA051E86A202D87F31BCA345DE 6ADF89180877B3EA3C92CAC5628D93CFB82AC3B522A724C81BFEE3846440D094 19ADFED40E822A6B6AB396F56FEEC0CEB71E26974DA436461E124A8D492567B2 E3C41C090BC689F6C78A5F5D32F6A1B96F4780BF993BF304C854665E16C2DD2D

Параметр	Значення (hex)
S_1	7CD7171360B20FB75DD9ECB68905E1B976CBAA92093106C5B843572B6D650656 D052068583390BF5AE1684908090D4BC87AF0472DF3B12A2E2C1A692453D4E6B B3BEC47F850F74C5EF908D3FE7F2968C3753F4A5A325961CFA2B0B66C7C07BA3 7DB9DC321D0D91FC82BBB44F36712253F21106FE5815ECBD3C53AC882EDADBFD
Ключ (сервер)	02917B7A817AF708207473B706E7DF8E1EB1C66E79F74D60AC03031EC35D7D3B

Сервер розшифрував k_1 та S_1 своїм секретним ключем і підтвердив підпис (Verification: true); отриманий ключ збігається з k (сервер виводить його з ведучим нулем).

7.7. Протокол: SendKey на сервері → ReceiveKey власною реалізацією (B). Для цього випадку сервер має бути відправником з меншим модулем, тому згенеровано новий ключ сервера 256 біт. На вкладці Send Key задано відкритий ключ абонента B (n_1, e_1); отримані ключ і підпис прийнято функцією receive_key із секретним ключем B та відкритим ключем сервера.





Параметр	**Значення (hex)**
n_s	817C9E2803023838D88B2754F43CF9F2A25FA0044A5D47F5A3864B7537B0C4AB
Key (k_1)	48DDC2F2E367FD2B5B99BFCEDD53437DFCB0969661C84D1F161095D3756BC73A79703A9532AC409817720572ED84C94B052F09CA0BF5CF81A2B9AB5E2353FEC3
Signature (S_1)	CE377F54EDC3DE74B6CBAA82982704AF34095ADF58C7B81D0AD09AD6B96FCEB4716C0409E3CD9565ABF52DB09A648B987D506D2DEE1EEE7FF11191C01F2D19DB
receive_key	7833F29341B17220

Функція receive_key прийняла ключ (підпис пройшов перевірку) і повернула *k* = 7833F29341B17220. Якщо перевіряти підпис за іншим модулем (замість відкритого ключа сервера), функція повертає None, тобто ключ відхиляється.

Таким чином, усі операції власної реалізації — шифрування, розшифрування, створення та перевірка підпису, відправлення й одержання ключа — коректно взаємодіють із тестовим середовищем в обох напрямках.

8. Особливості реалізації та шляхи їх розв'язання

Узгодження розмірів модулів. Для протоколу необхідно $n_1 \geq n$, інакше підпис $S < n$ може не вміститися в модуль абонента B. Тому після генерації двох пар їх міняють місцями, якщо $pq > p_1q_1$.

Розміри модулів при перевірці з сервером. Протокол вимагає $n_{\text{відправника}} \leq n_{\text{одержувача}}$, тому з одним серверним ключем перевірити обидва напрямки неможливо: для ReceiveKey потрібен серверний ключ більший за 512 біт, а для SendKey — менший. Тому на сервері генерувались два ключі (1024 і 256 біт).

Відкидання виродженого випадку p = q. У разі збігу двох простих n було б повним квадратом і легко розкладалось. Функція pair повторно генерує q, доки воно не відрізнятиметься від p.

Обмеження на відкриту експоненту. Значення $e = 65537$ є простим, тому $\gcd(e, \varphi(n)) \neq 1$ лише якщо e ділить $\varphi(n)$; для цього випадку передбачено запасний вибір випадкового e, що задовольняє умову.

Демонстрація відхилення складеного числа. Для простого числа тест завжди завершується відповіддю «просте», тому для демонстрації відхилення додатково розглянуто складене число, що проходить пробні ділення (п. 2).

Обмеження навчальної реалізації. Реалізовано «підручникове» RSA: повідомлення шифрується і підписується безпосередньо, без доповнення (OAEP/PSS) і без попереднього гешування; випадкові числа беруться з вбудованого модуля random, який не є криптографічно стійким. Це відповідає вимогам завдання, але для реального використання непридатне. Також не перевіряється наявність великих простих дільників у $p - 1$ і $q - 1$ (рекомендація щодо «гарних» простих чисел).

Структура секретного ключа. Секретний ключ зберігається як (d, p, q), але функції Decrypt і Sign використовують із нього лише d та добуток $pq = n$. Числа p і q знадобилися б для прискорення за китайською теоремою про лишки, яка тут не застосовувалась.

<u>Відповіді на контрольні питання:</u>

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

По-перше, кандидат формується з виставленими старшим бітом (довжина числа рівно 256 біт, а отже, $n = pq$ має 512 біт) і молодшим бітом (непарність). По-друге, виконуються пробні ділення на 25 малих простих: це дешево і відсіює ≈ 76 % кандидатів до дорогих обчислень. По-третє, у тесті Міллера–Рабіна для кожної основи перевіряється $\gcd(x, n) = 1$ (знайдений спільний дільник одразу доводить складеність) і умова сильної псевдопростоти; $k = 20$ раундів зменшують

ймовірність помилки до 4^{-20}. По-четверте, під час побудови пар перевіряється $p \neq q$ та впорядкування $pq \leq p_1 q_1$, потрібне для протоколу.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

Мають виконуватися: p і q — різні прості; $n = pq$; $\varphi(n) = (p-1)(q-1)$; $1 < e < \varphi(n)$ та $\gcd(e, \varphi(n)) = 1$; $ed \equiv 1 \pmod{\varphi(n)}$. У main.py виведено перевірки: простота p і q за тестом Міллера–Рабіна з $k = 40$ (True, True), $n = pq$ (True), $\gcd(e, \varphi) = 1$ (True), $e \cdot d \bmod \varphi = 1$ (True). Крім того, перевірено, що розшифрування шифртексту та перевірка підпису повертають вихідне повідомлення для обох абонентів. Перевірку наявності великих простих дільників у $p - 1$ і $q - 1$ не виконано.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Encrypt: повідомлення M і відкритий ключ адресата (n, e). Decrypt: криптограма C і секретний ключ власника (d, n). Sign: повідомлення M і секретний ключ відправника (d, n). Verify: повідомлення M, підпис S і відкритий ключ відправника (n, e). Кожна операція є піднесенням до степеня за модулем n, тому їй потрібні лише відповідна експонента та модуль. Операції з відкритими ключами не повинні потребувати секретних даних, а секретні операції не потребують $\varphi(n)$, p та q (вони потрібні лише для побудови ключів), і їх розкриття означало б компрометацію системи.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?

Конфіденційність забезпечується кроками 2 і 4: k і підпис S шифруються відкритим ключем B, тому прочитати їх може лише B із секретним d_1. Підтвердження справжності відправника забезпечується кроками 3 і 7: підпис $S = k^d \bmod n$ створюється секретним ключем A, а B перевіряє його відкритим ключем A. Умова $n_1 \geq n$ забезпечує коректне шифрування підпису.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Одержувач B розшифровує k_1 та S_1 своїм секретним ключем d_1, після чого перевіряє, що $S^e \bmod n = k$ за відкритим ключем A; лише тоді k приймається. Якщо перевірка не пройдена, ключ відкидається (функція повертає None). Також доцільно перевіряти, що $0 < k < n$, а відкритий ключ A справді належить A (наприклад, через сертифікат): сам протокол цього не гарантує. Захисту від повторного надсилання перехопленого повідомлення протокол також не має.

Висновки:

У ході виконання комп'ютерного практикуму було реалізовано криптосистему RSA та протокол розсилання ключів із підтвердженням справжності відправника. Реалізовано швидке піднесення до степеня за схемою Горнера, розширений алгоритм Евкліда, тест Міллера–Рабіна з пробними діленнями та генерацію простих чисел довжиною 256 біт. Для чотирьох згенерованих чисел було відкинуто 385 кандидатів; на вибірці з 300 простих кандидатів на одне просте число було в середньому 90,0 (теоретично ≈ 88,7), пробні ділення відсіяли 76,0 % кандидатів (теоретично ≈ 75,9 %), а генерація одного простого числа тривала в середньому 7,9 мс. Побудовано ключові пари абонентів A і B з 512-бітними модулями та $e = 65537$; перевірено співвідношення $n = pq$, $\gcd(e, \varphi(n)) = 1$, $ed \equiv 1 \pmod{\varphi(n)}$. Шифрування, розшифрування, підпис та його перевірка працюють коректно для обох абонентів, а в протоколі розсилання ключів абонент B отримав те саме значення k, яке надіслав A; при спотвореному підписі або хибному ключі відправника ключ відхиляється. Слабкості навчальної реалізації: відсутні доповнення (OAEP/PSS) та гешування повідомлення, використовується некриптографічний генератор випадкових чисел, не перевіряється структура $p - 1$ та $q - 1$. Взаємодію з тестовим середовищем перевірено в обох напрямках для шифрування, розшифрування, підпису, його перевірки та протоколу розсилання ключів: усі шість перевірок успішні, а для протоколу використано ключі сервера 1024 та 256 біт відповідно до умови «модуль відправника не більший за модуль одержувача».